PREDICTING TERM DEPOSIT SUBSCRIPTION USING MACHINE LEARNING

CHIRATH SETUNGE

Table of Contents

GitHub Link

Introduction

The report documents the complete development of a classification model to predict the likelihood of a client subscribing to a term deposit. The prediction is based on their demographic, social, and financial data. The "Bank Marketing" dataset is used for this analysis, and it was taken from the UC Irvine Machine Learning Repository. The classification task was accomplished using two models: Random Forest, an ensemble learning method, and the Neural Network deep learning approach.

Dataset Overview

The "**Bank Marketing**" dataset used in this project is **bank-full.csv.** It contains information about a direct marketing campaign conducted by a Portuguese banking institution. The dataset consists of **45,211** instances and **17** features with a mix of categorical and numerical features, The target variable, y, indicates whether a client subscribed to a term deposit. This classification task belongs to the **business domain** and involves multivariate data analysis. Forty-five thousand two hundred eleven instances were recorded between May 2008 and November 2010.

Corpus Preparation

Data Exploration and Understanding

The Exploratory Data Analysis (EDA) is a crucial step in the data exploration and understanding of the project domain and the dataset.

The features are divided into four parts as the first step by understanding the meta information from the UC Irvine Machine Learning Repository.

- Client Data: age, job, marital, education, default, housing, loan, balance
- Contacts of the current campaign: contact, month, day, duration
- Others: campaign, pdays, previous, poutcome
- Target variable: y

Confirming Dataset Shape: Identify the number of rows(instances) and columns(features) that are present in the dataset to ensure the intended dataset, bank-full.csv, is loaded, distinguishing it from the other three datasets (bank.csv, bank-full.csv, bank-additional.csv, and bank-additional-full.csv).

- The number of rows in the dataset is 45211
- The number of columns (features) in the dataset is 17

Identifying Data Types: All the data types in the dataset are either numerical or categorical. The features data types are as follows.

```
The data types of the columns (features) in the dataset a
age           int64
job          object
marital      object
education    object
default      object
balance       int64
housing      object
loan         object
contact      object
day           int64
month        object
duration      int64
campaign      int64
pdays         int64
previous      int64
poutcome     object
y            object
dtype: object
```

Identify Missing Values: All the features are checked using *isnull()* methods to identify missing values. Fortunately, there are no missing values, so there is no need to fill any missing values under data cleaning.

```
The number of missing values in the dataset is:
age          0
job          0
marital      0
education    0
default      0
balance      0
housing      0
loan         0
contact      0
day          0
month        0
duration     0
campaign     0
pdays        0
previous     0
poutcome     0
y            0
dtype: int64
```

Identify Duplicate Records: None of the records are duplicated, and it was checked using *duplicated()* method.

```
The number of duplicate rows in the dataset is: 0
```

Identify Categorical Features: Extract the categorical features from the dataset, excluding the target column. The identified categorical features are

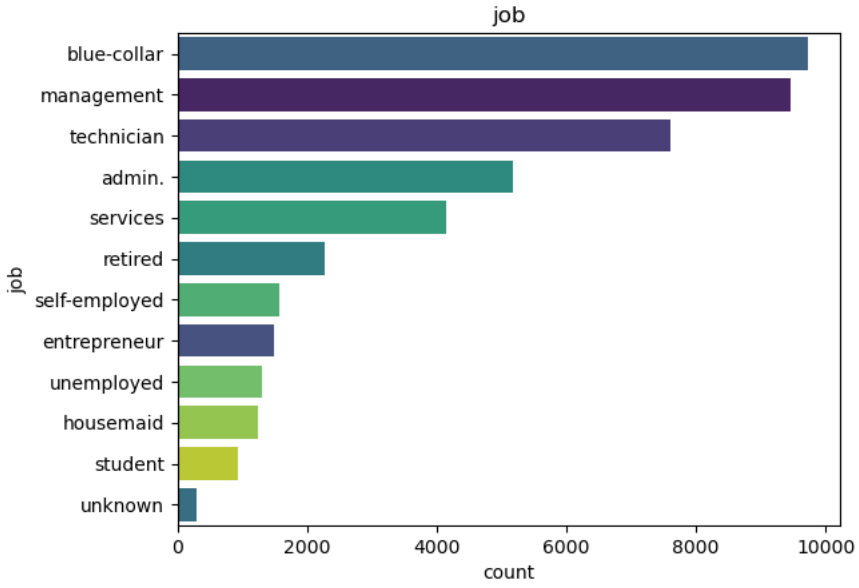
- The categorical features in the dataset are (except the target column (y)):
- ['job', 'marital', 'education', 'default', 'housing', 'loan', 'contact', 'month', 'poutcome']

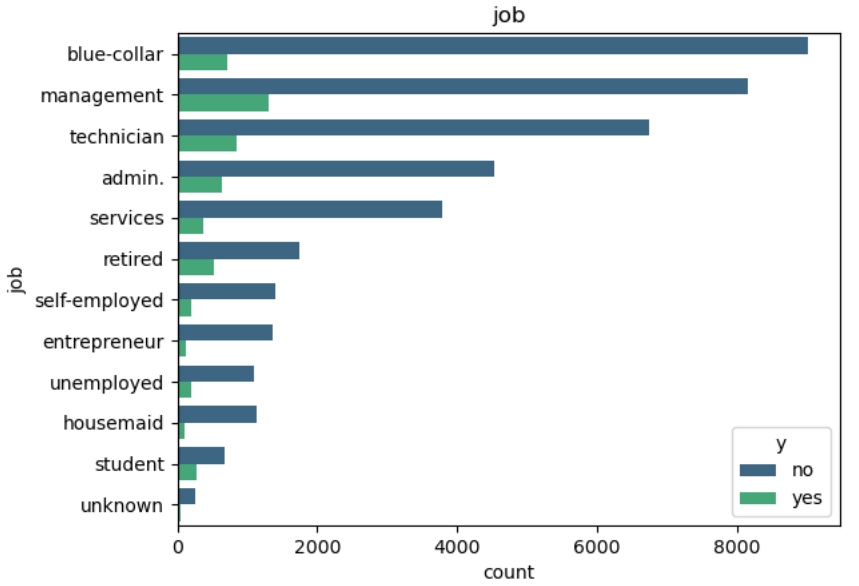
Unique Values in Categorical Features:

job	management, technician, entrepreneur, blue-collar, unknown, retired, admin., services, self-employed, unemployed, housemaid, student	12
marital	married, single, divorced	3
education	tertiary, secondary, unknown, primary	4
default	yes, no	2
housing	yes, no	2
loan	yes, no	2
contact	unknown, cellular, telephone	3
month	may, jun, jul, aug, oct, nov, dec, jan, feb, mar, apr, sep	12
poutcome	unknown, failure, other, success	4

Distribution of the Categorical Features with Respect to the Target Column:

Job



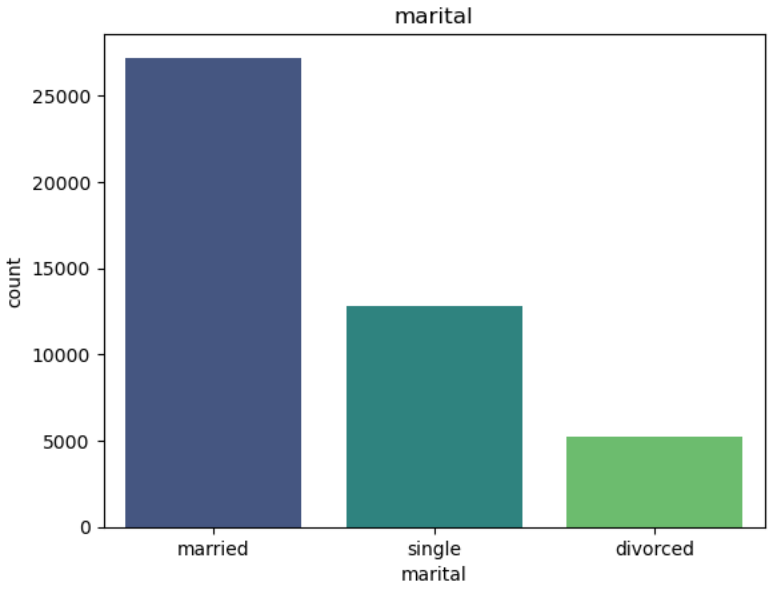


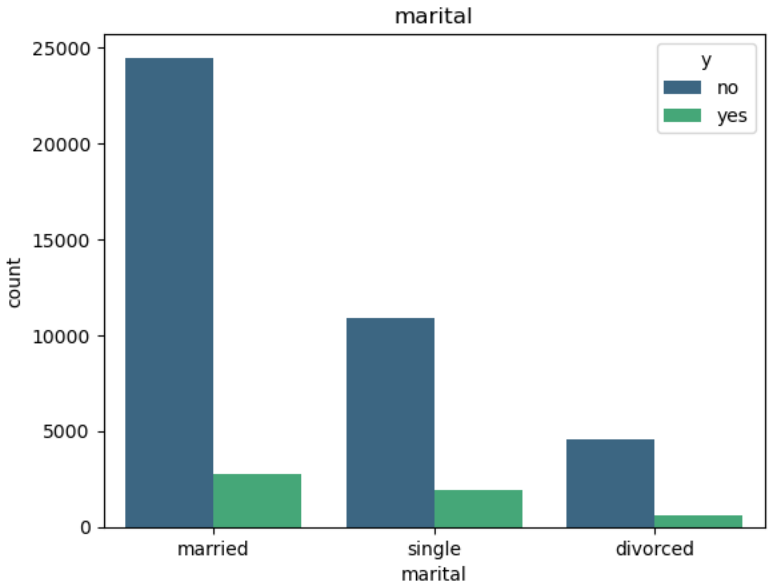
- The most common jobs in the dataset are blue-collar, management, and technician, indicating that the campaign is primarily targeted at individuals in these professions. The student category is the lowest verified job type, and it suggests that banks are less focused on students, but interestingly, students subscribe to term deposits more than entrepreneurs,

unemployed, self-employed, and housemaids, even though banks focus on them more than students.

- Retired clients are more likely to say yes to the term deposit. This could be due to factors such as financial stability or a preference for low-risk investments during retirement.

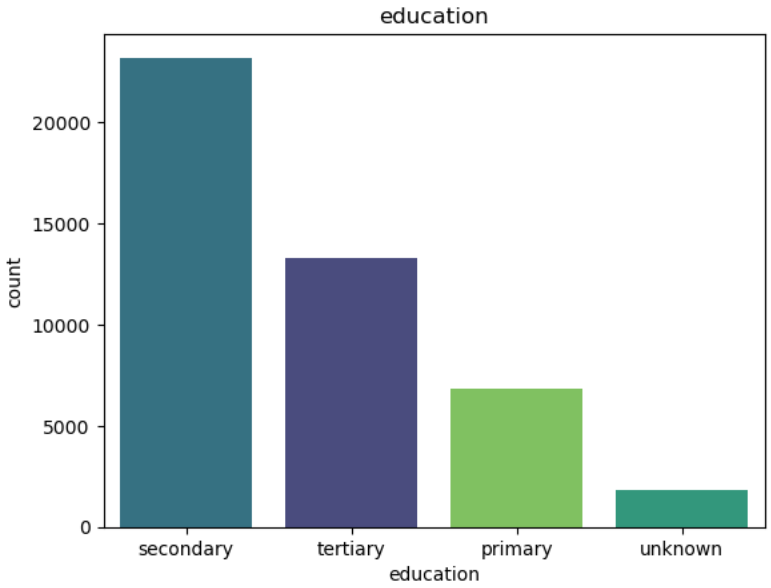
Marital

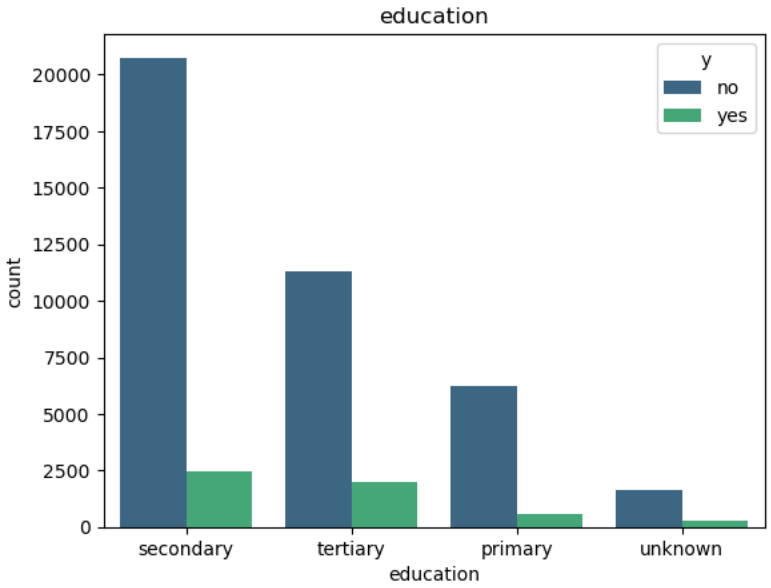




- Married people are more frequent in the dataset, on the other hand, divorced people are least frequent in the dataset. Married clients show higher subscription rates in proportion to their higher count, but interestingly, single clients are more likely to say yes to the term deposit. It indicates that single clients are likely to invest in low-risk investments.

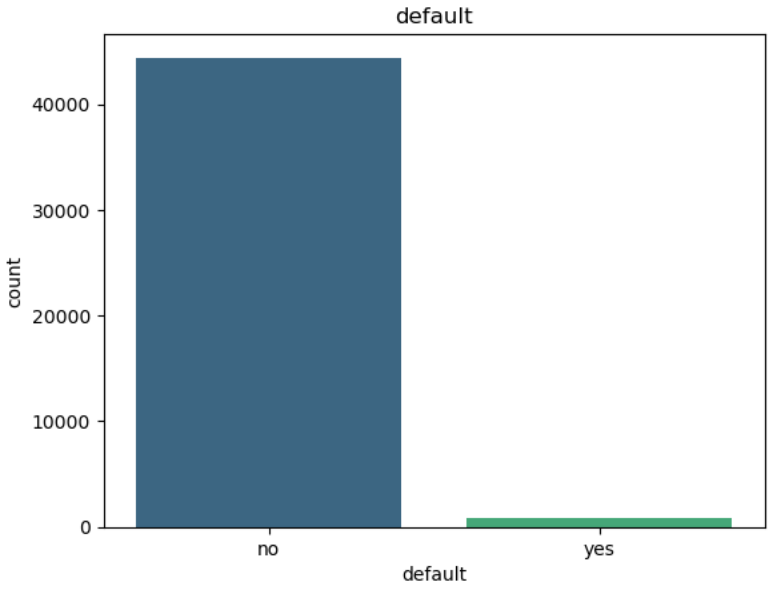
Education

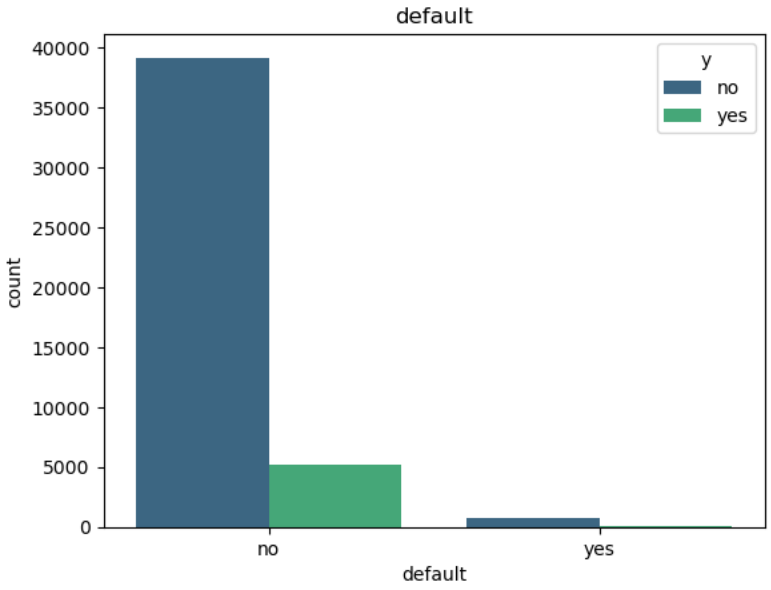




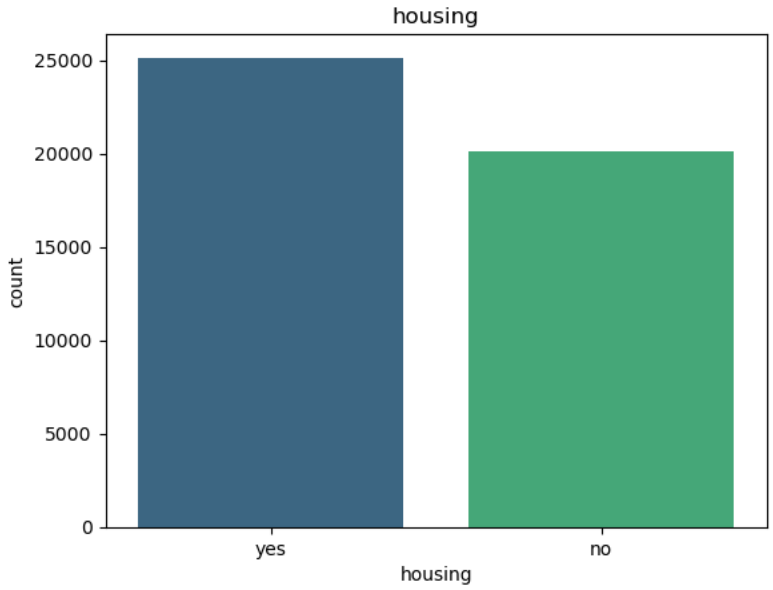
- The education category is an ordinal data type, and here, it clearly indicates that education level significantly impacts saying yes to the term deposit. Tertiary education level clients have the highest level of acceptance rate, proportion to their count. Suggesting that higher education levels may correlate with better financial awareness

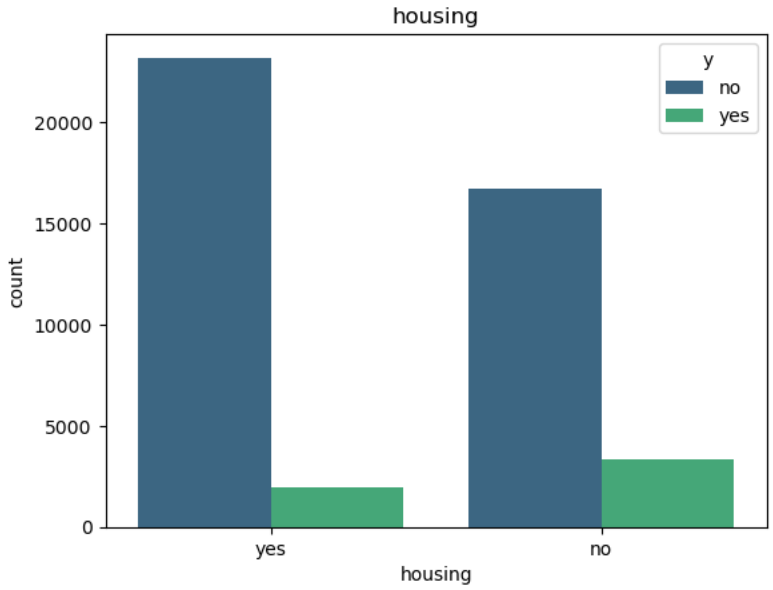
Default





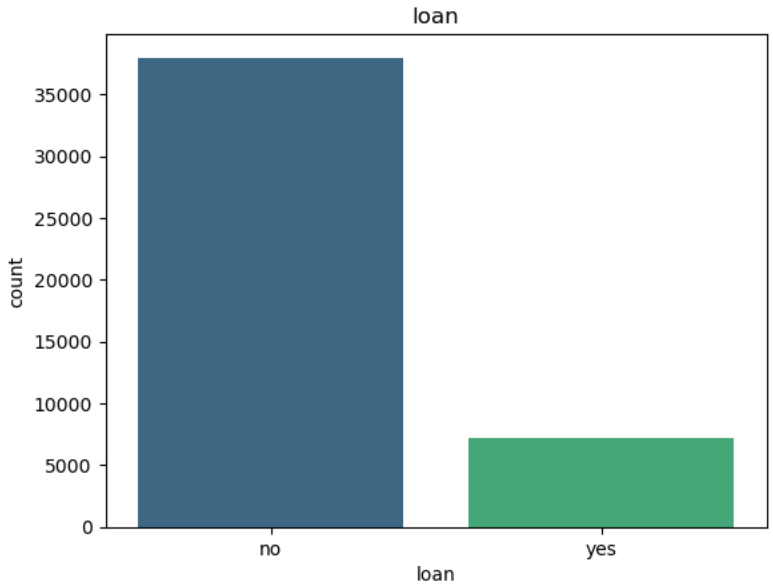
Housing

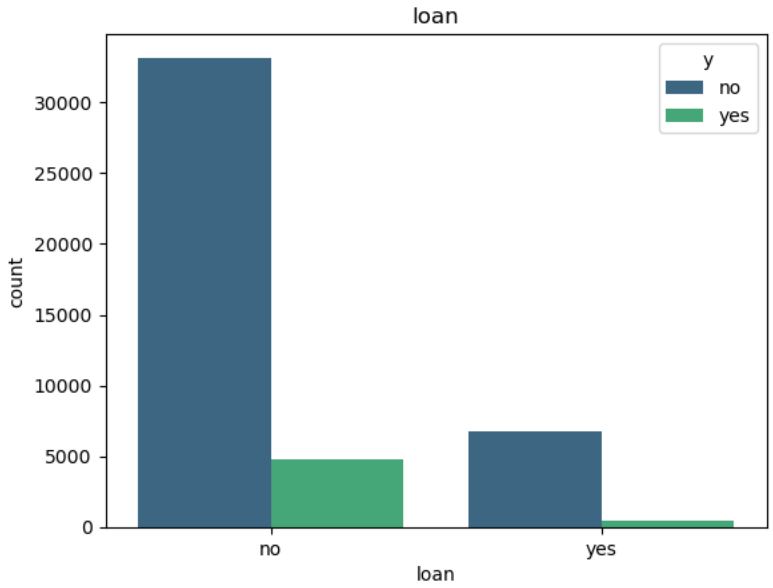




- House loans also have a significant impact on responding to the term deposit. The clients who have no housing loan tend to subscribe to the term deposit more than the clients who have a house loan.

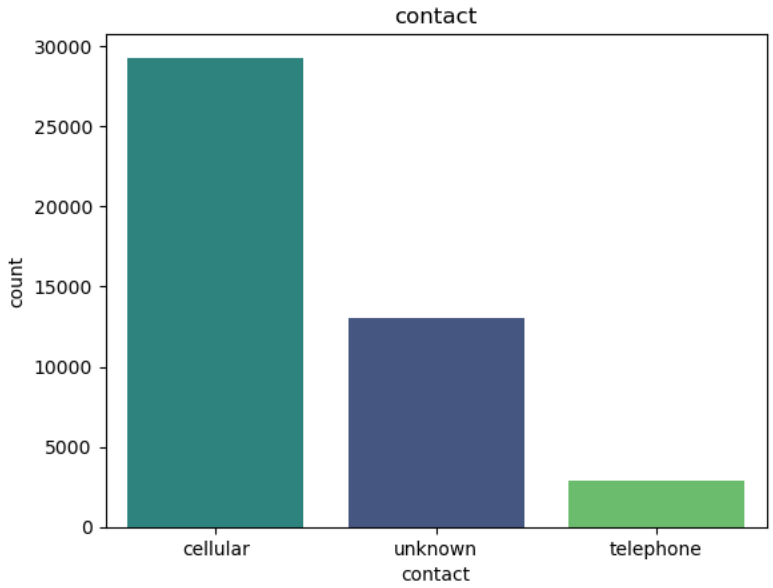
Loan

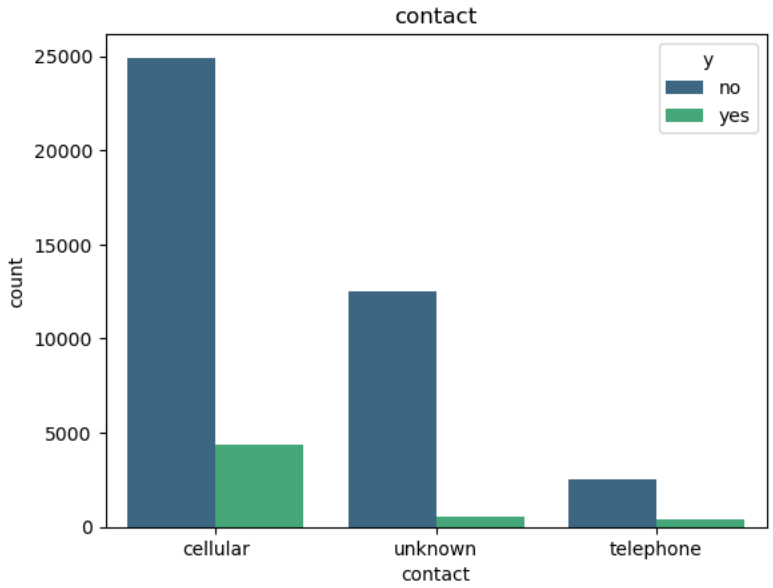




- It goes the same way that clients who don't have a personal loan tend to accept the term deposit compared to clients who have a personal loan.

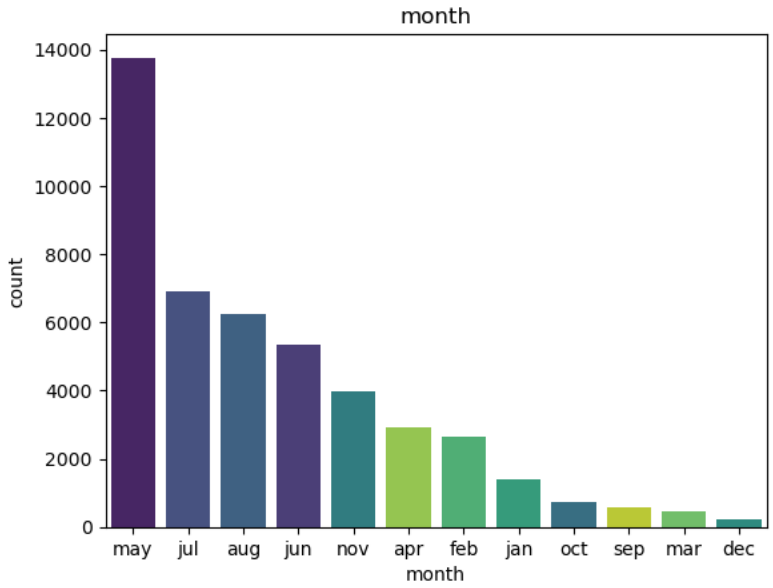
Contact

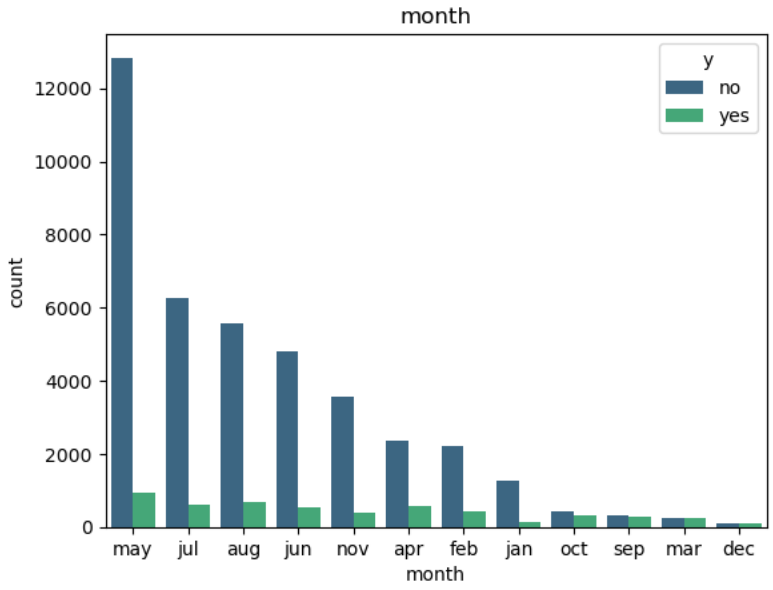




- In the modern world, people are more likely to use cellular devices to contact so the bank also used cellular mode to contact customers, and it was the most successful method to convince the clients to subscribe to the term deposit, and it's evident in the figures.

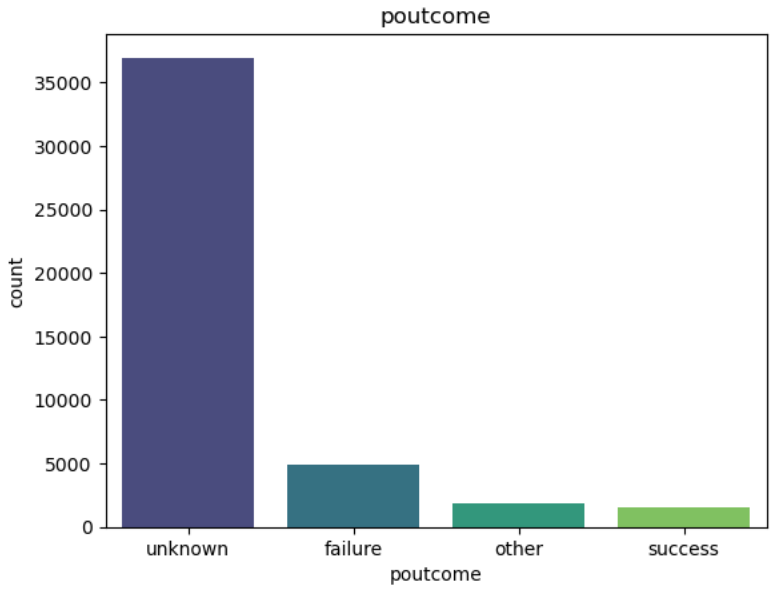
Month

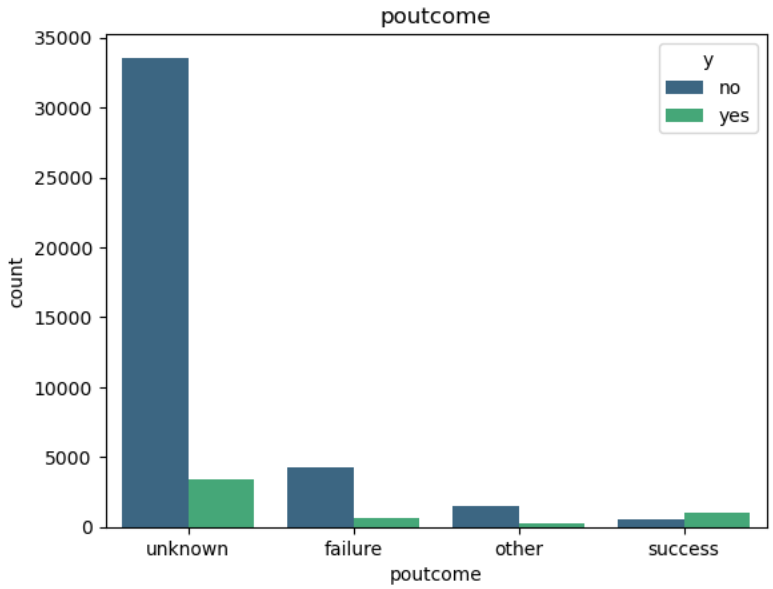




- The top three months that banks are more focused on improving the campaign are May, July, and August it is indicated that banks are trying to turn clients' summer expenses into term deposits, and it shows significant improvements, but clients tend to subscribe to term deposit on March, December, September, October in order after the summer maybe they are using their summer savings.

Poutcome

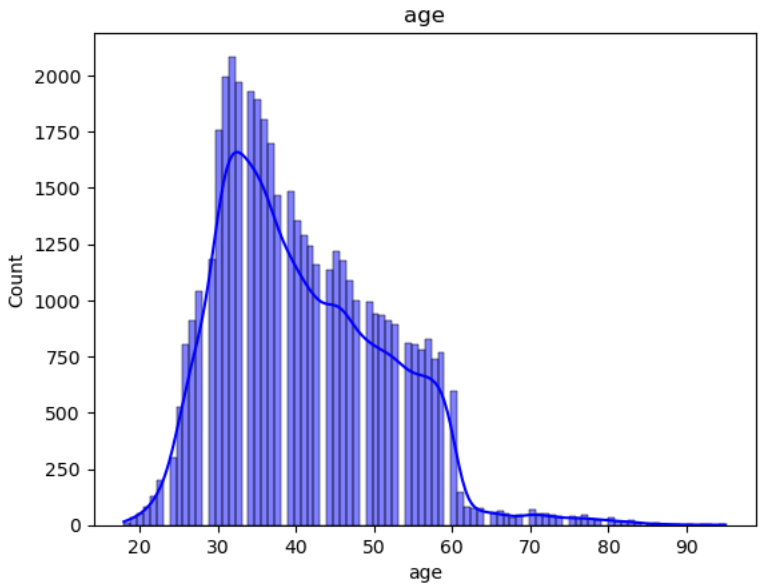




- In the entire dataset, the majority of the client's previous campaigns' output is unknown. Interestingly, the clients who have successful outcomes from the previous campaign are more likely to once again subscribe to the term deposit.

Distribution of the Numerical Features:

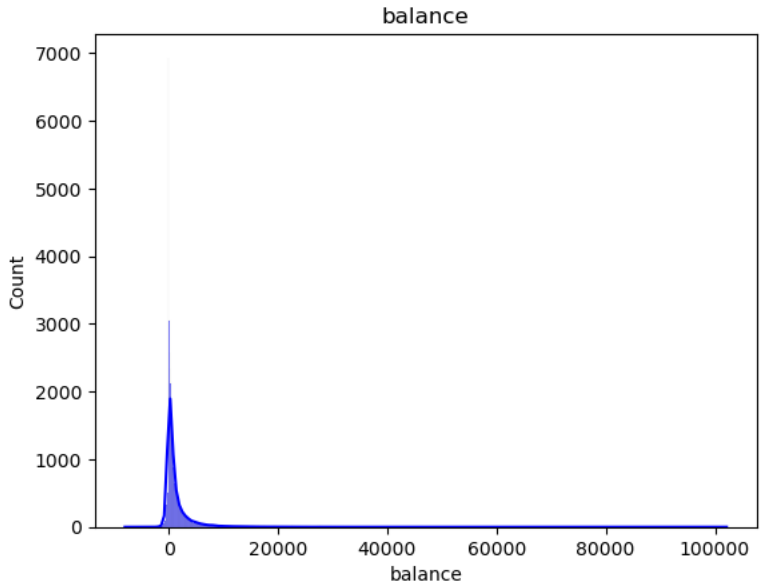
Age



count 45211.000000
mean 40.936210
std 10.618762
min 18.000000
25% 33.000000
50% 39.000000
75% 48.000000
max 95.000000
Name: age, dtype: float64

- The average age is 41 with a median of 39, indicating a very small right-skew in the data, other than that, the age feature has a perfect normal distribution till age 65. The age ranges from 18 to 95, and most age records are concentrated below age 65.

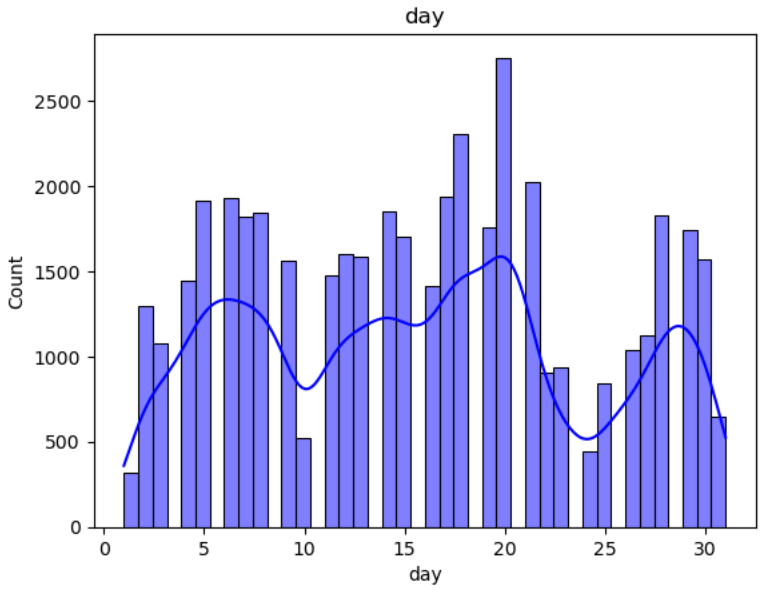
Balance



count 45211.000000
mean 1362.272058
std 3044.765829
min -8019.000000
25% 72.000000
50% 448.000000
75% 1428.000000
max 102127.000000
Name: balance, dtype: float64

- The visualization and the descriptive statistics indicate a huge right-skew in the data due to few clients with very high balances and values, evident that the average is 1362 and the median is 448. Interestingly, the balance increases the chance of the client subscribing to the term deposit is high. The balance feature has a few very high records.

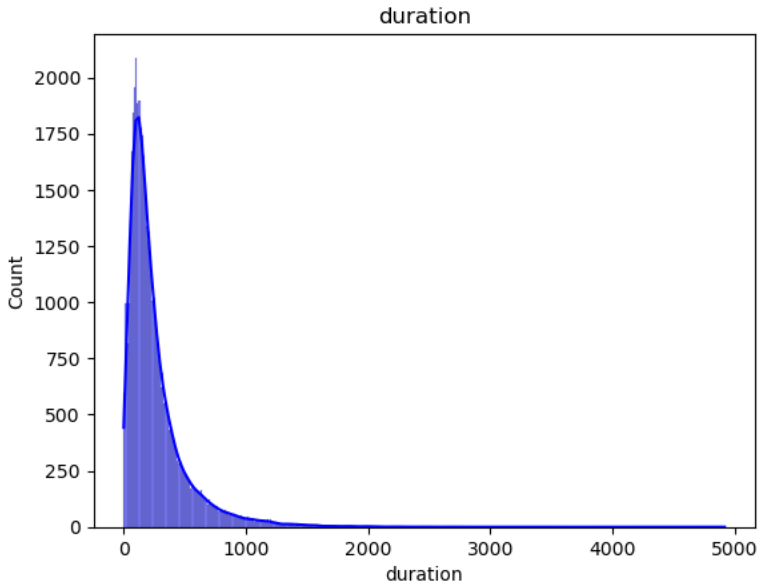
Day



count 45211.000000
mean 15.806419
std 8.322476
min 1.000000
25% 8.000000
50% 16.000000
75% 21.000000
max 31.000000
Name: day, dtype: float64

- The visualization and descriptive statistics don't show any abnormality in the day feature and indicate that banks are more focused on promoting the campaign on **week 1 and week 3** and in the last week of the month.

Duration

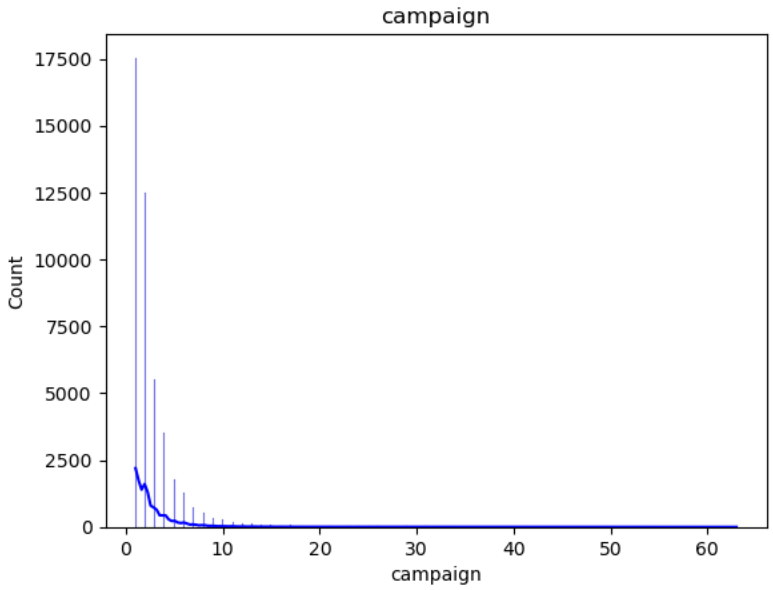


count	45211.000000
mean	258.163080
std	257.527812
min	0.000000
25%	103.000000
50%	180.000000
75%	319.000000
max	4918.000000

Name: duration, dtype: float64

- The contact duration for each client ranges from 0 to 4918 seconds. The average duration is 258, with a median of 180, indicating a right-skew in the data.

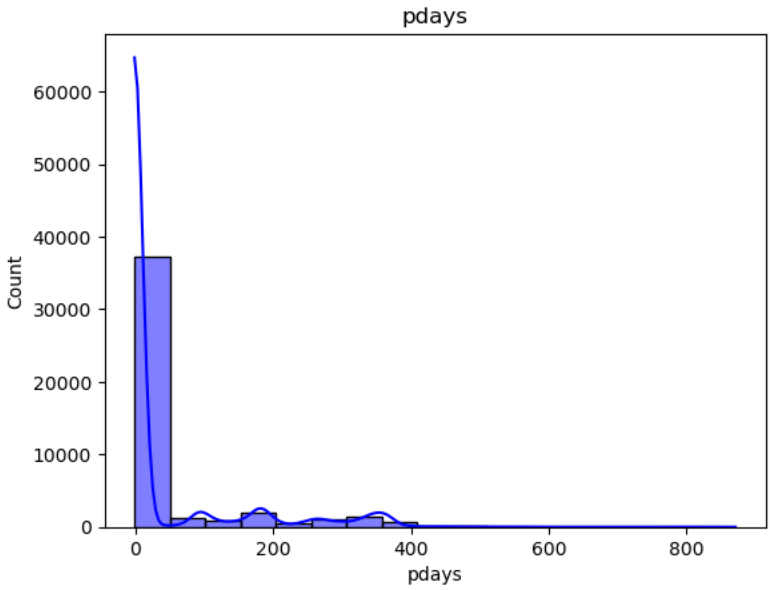
Campaign



count 45211.000000
mean 2.763841
std 3.098021
min 1.000000
25% 1.000000
50% 2.000000
75% 3.000000
max 63.000000
Name: campaign, dtype: float64

- The majority of the clients have contacted 0 to 3 times in the campaign, and there are a few extreme values like 63 that need to be handled.

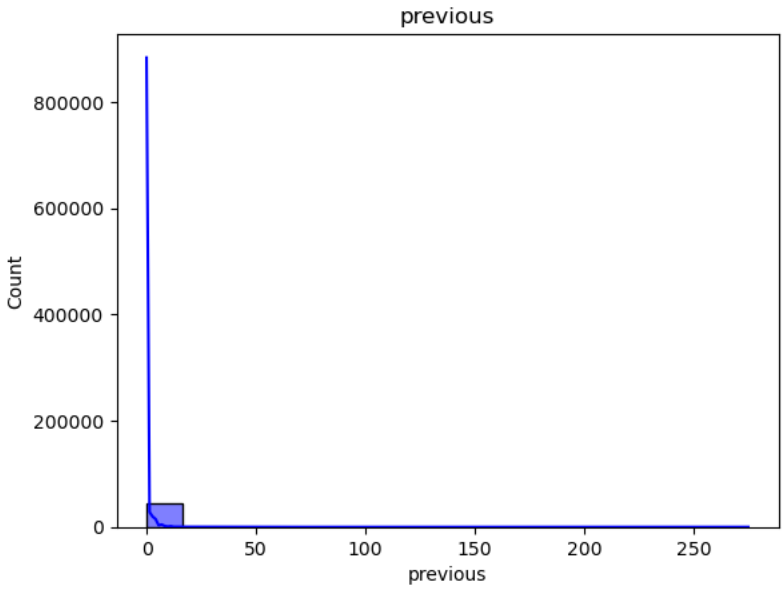
Pdays



count 45211.000000
mean 40.197828
std 100.128746
min -1.000000
25% -1.000000
50% -1.000000
75% -1.000000
max 871.000000
Name: pdays, dtype: float64

- The pdays feature (number of days that passed by after the client was last contacted from the previous campaign) is highly concentrated with the value -1 (client was not previously contacted) even the 75 percentile also has -1. The average of 40 and median of -1 indicate that the pdays column is highly right-skewed.

Previous



count 45211.000000
mean 0.580323
std 2.303441
min 0.000000
25% 0.000000
50% 0.000000
75% 0.000000
max 275.000000
Name: previous, dtype: float64

- The previous feature (how many times the client was contacted before this campaign). The average of 0.5 and median of 0 indicate the slight left-skew in the data, but there are few high extreme values that need to be addressed.

Data Cleaning

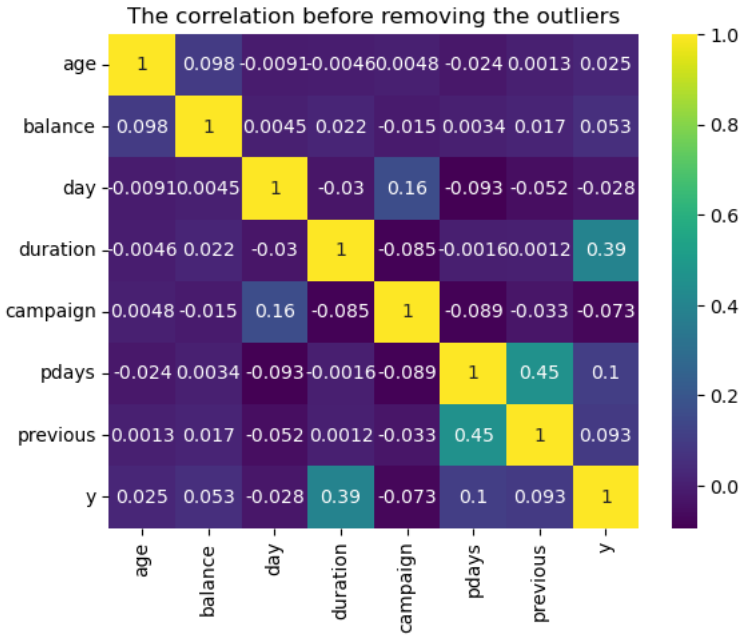
Data cleaning is a most important part of the data science dataset preparation process; it ensures quality and reliability. It involves removing or correcting inconsistent and inaccurate data that could affect the model's performance.

In the "Bank Marketing" dataset there are no missing values were observed and no duplicates were found, this ensures that the dataset is free of redundancy and need of imputation.

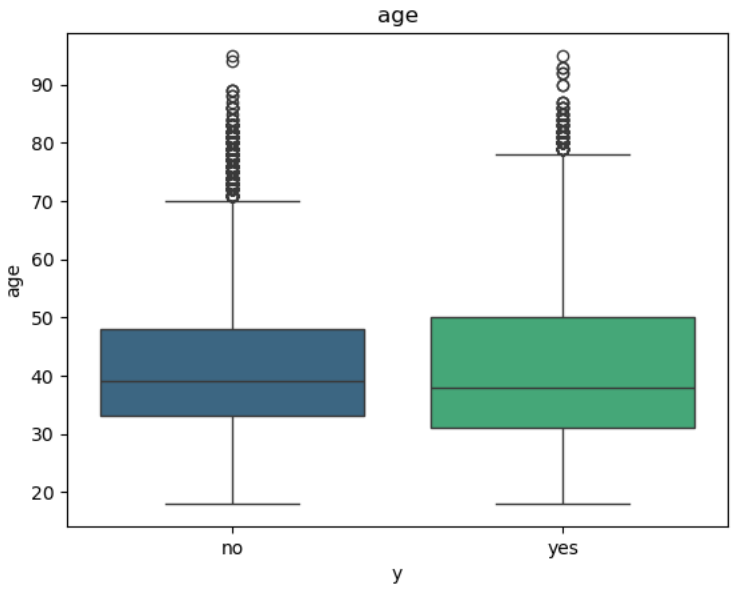
Outlier Identification and Handling

To identify the extreme values that deviate from the majority of the data in each numerical feature, the numerical features were analyzed and handled the outlier effectively.

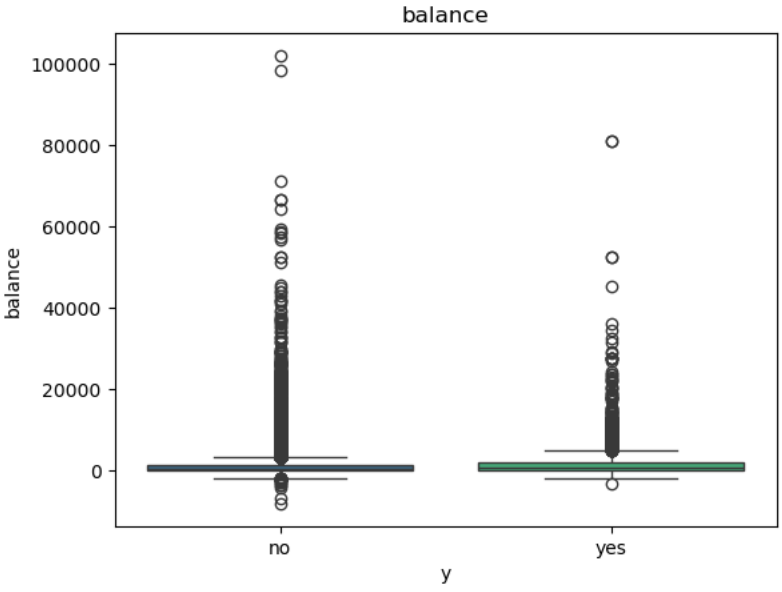
A correlation matrix was generated and asses the relationship between numerical features and the target variable.



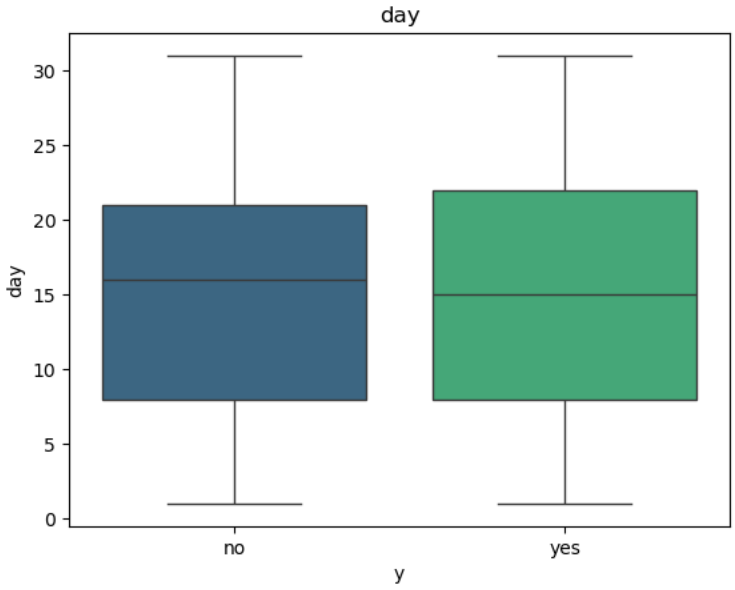
Then, each numerical feature is visualized using a boxplot with respect to the target variable to identify the extreme values that are present and to evaluate their impact.



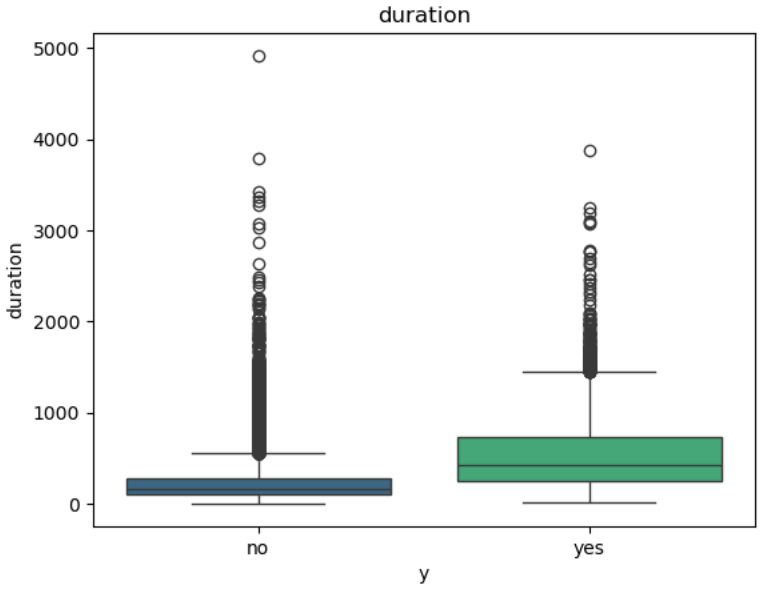
- None of the outliers were managed in the age feature because it doesn't show any extreme outliers, and the project task is classification, and the outliers are balanced in the target variable.



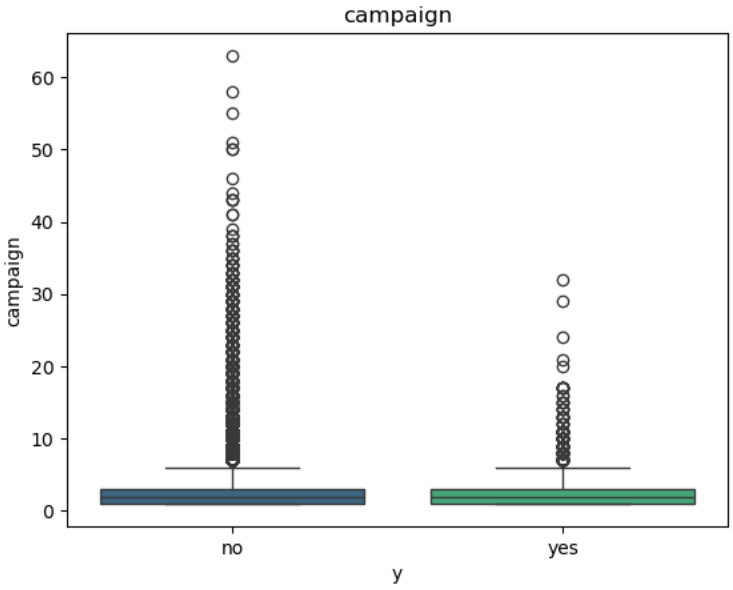
- To handle the extreme outliers that are above the 80000 winsorization limit is set to top 0.01



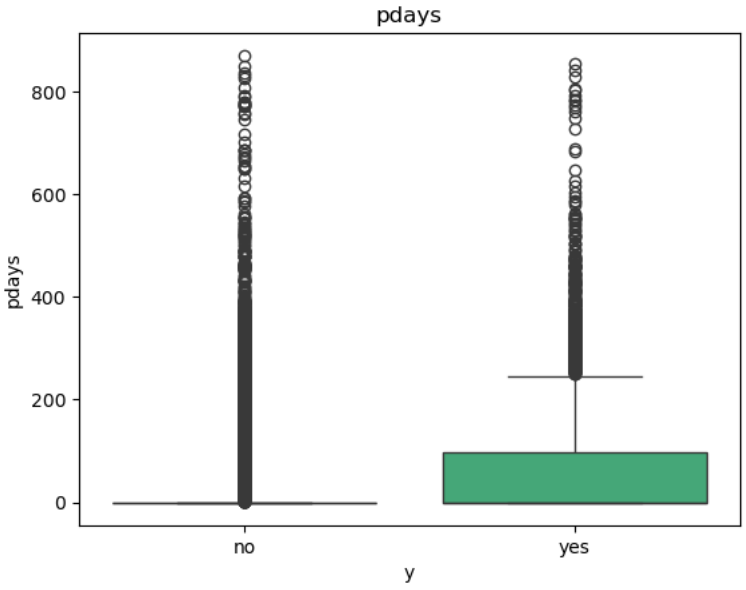
- No changes have made to day feature

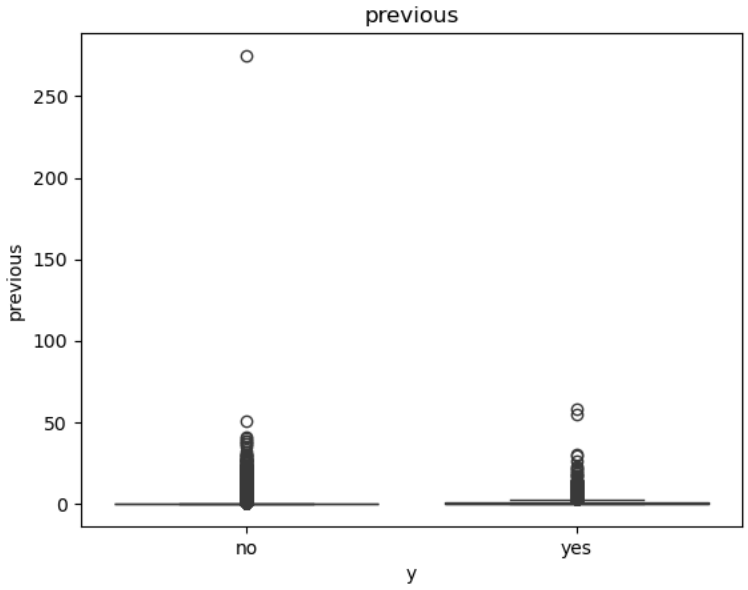


- Duration numerical feature is important and highly correlated. In order to improve that correlation apply the winsorization to top 0.005 percent to handle extremely high points.



- After 40 the there are few less saturated outliers, in order to handle the apply the winsorization with the limit set to top 0.005

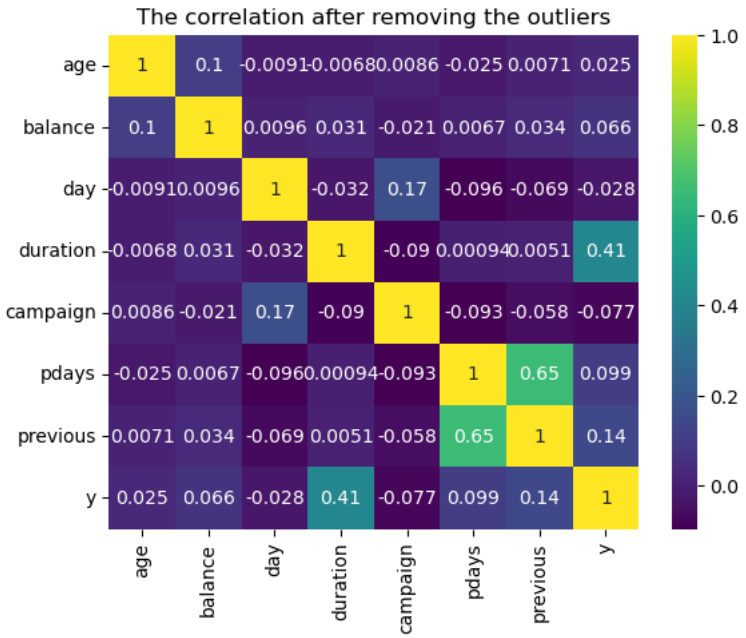




- For both pdays and previous numerical features that are highly correlated with each other and they represent the same information different way so apply the winsorization to both to top 1% to increase correlation more.

The winsorization method is used to address the outliers that are present in numerical features, this method is specially selected because it limits extreme values without removing data. It also ensures minimal data loss while keeping the influence of extreme values. Different limits were applied to features based on their specific distribution and extent of their outliers

A new correlation matrix was generated after the winsorization method to evaluate the results: increased correlation of valuable feature such as duration, balance, previous with the target variable, strengthening the predictive power. Decreased correlation for irrelevant features such as pdays. Helping to refine the dataset for classification.



Data Transformation

In machine learning models like Random Forest and Neural Networks respond differently to raw and transformed data

Random Forest does not strictly require scaling or normalization, as it ensemble methods that operates on decision trees that split based on feature thresholds. Encoding categorical features are important

Neural Networks are very sensitive to the numerical feature scaling and categorical feature encoding since they compute the weighted sums and activations.

Encoding Categorical Features

The nature of the categorical features are identified in the EDA process and for encoding purpose they were categorized as ordinal and nominal features.

Ordinal Features	education
Nominal Features	job, marital, default, housing, loan, contact, month, poutcome

Ordinal Features	Ordinal encoding
Nominal Features	One-Hot encoding

The ordinal encoding applied to the education feature; to highlight its hierarchal nature and for all the other categorical features, one-hot encoding was used to give fair categorical representation.

Scaling Numerical Features

Neural networks are highly sensitive to the magnitude of the input values. This highlights the importance of scaling the numerical features. The StandardScaler was used to standardize numerical features by eliminating the mean and scaling to unit variance, making the data more suitable for neural networks.

The scaler was fitted and transformed using *fit_transform* to compute the mean and standard deviation of the training data and then standardize it. On the other hand, for the test set, the scaler was only transformed using *transform,* applying the previously computed statistics from the training set to ensure consistency and prevent overfitting and data leakage.

Addressing Class Imbalance

In the "Bank Marketing" dataset, there is a huge class imbalance in the target column, and the task is also a classification task. Imbalanced features can lead to biased model predictions, where the model prefers the majority class. Addressing this imbalance is crucial for improving the model's performance.

No	39,922 records
Yes	5,289 records

The technique used to address the class imbalance is the Synthetic Minority Oversampling Technique (SMOTE) it generates synthetic data for the minority class by techniques called interpolating between existing data points. This process ensures that the training dataset is balanced without duplicating or removing records.

Only the training dataset is balanced to simulate real-world conditions; this helps to evaluate the model's ability to handle imbalanced data during testing, and it helps models to be trained to recognize the minority class effectively.

Undersampling was avoided because the class imbalance is significant. Eliminating huge numbers of data from the majority class would lead to a huge loss of valuable data, which could also lead to a reduced model's learning capacity.

Dataset Split

To evaluate the machine learning models and ensure the models performed well with new unseen data, the dataset is split into training and testing sets. Using the *train_test_split* function. 80% of the dataset is the training set, and 20% of the dataset is a testing set. The *stratify* parameter was set to the target variable y to maintain the same class distribution in both training and testing sets. This was done to handle the class imbalance in the dataset.

Solution Methodology

Overview of Machine Learning models

In this project, To handle the classification task, the "Bank Marketing" dataset problem, the main machine learning models were built. Random Forest and Neural Networks.

Random Forest

Random Forest is an ensemble learning method, in other work words it's a combining model approach, in here multiple decision trees are combined. Random Forest comes under the bagging type. During training, it builds a forest of decision trees on various sub-samples of the dataset, and during the prediction, it aggregates the results using majority voting from all the trees to make the final classification.

Random Forest reduces overfitting by averaging the output of multiple trees. It can also identify the feature contribution and more interpretability of the results. Robustness for classification tasks that also resistance to noise. It can also capture non-linear relationships in the data which is crucial for the project's classification task.

Neural Networks

Neural Networks consist of interconnected layers of neurons that learn and process data. The neural networks learn patterns by iteratively updating weights using the backpropagation technique.

Neural Networks are good at finding complex and non-linear relationships between features and target variables. They can scale well with larger datasets. By using regularization techniques and optimization algorithms, neural networks can generalize well to unseen data.

Model Design and Implementation

Random forest

The model was implemented using the *sklearn* library. When building the model, the model parameter plays an important role in improving the model's performance. GridSearchCV was used for hyperparameter tuning. This method is an exhaustive testing combination of parameters such as the number of estimators, maximum tree depth, and minimum samples required to split. Through cross-validation, the best configuration is determined, ensuring a well-balanced model. The dataset is highly imbalanced, so the *class_weight* parameter is set to 'balanced_subsample," which will adjust the weights based on the target variables' distribution.

Neural Networks

The neural network was implemented using TensorFlow and Keras with a sequential model structure; it consists of A dense layer with 100 neurons with a ReLu activation function and a second dense layer with 50 neurons and a ReLu activation function. The output layer uses a sigmoid activation function compared to other layers' ReLu activation function because this is a binary classification task. The Adam optimizer was selected for efficient gradient descent, and the loss function was set with binary cross entropy to match the classification task. Various parameter configurations were tested with early stopping to find the best model with the best scores and best architecture.

Evaluation Criteria

Random Forest

Classification Report

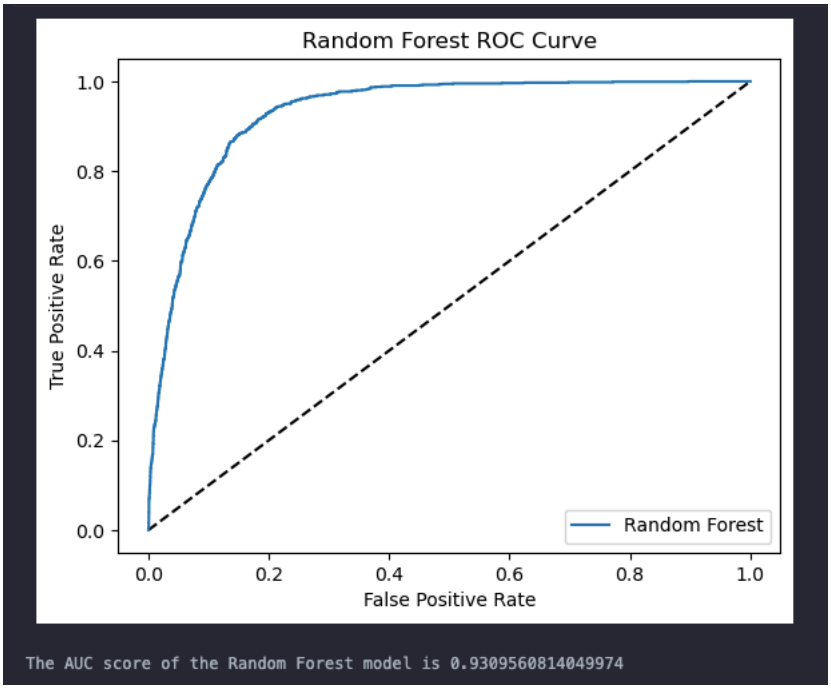
```
The accuracy of the Random Forest model is 0.90755280327325
The confusion matrix of the Random Forest model is
[[7658  327]
 [ 509  549]]
The classification report of the Random Forest model is
              precision    recall  f1-score   support

           0       0.94      0.96      0.95      7985
           1       0.63      0.52      0.57      1058

    accuracy                           0.91      9043
   macro avg       0.78      0.74      0.76      9043
weighted avg       0.90      0.91      0.90      9043
```

The Random Forest model demonstrated strong performance in the classification task, achieving an accuracy of 90.75% on the testing dataset.

The achieved precision, recall, and F1-scores indicate the model is highly effective at identifying instances of the majority class, but the performance for the minority class is lower due to the testing dataset not being balanced like the training set, so the class imbalance affects the model results.

True Negative	7658
False Positive	327
False Negative	509
True Negative	549

ROC-AUC



The high AUC accuracy value of 0.93 indicated that the random forest model performs strongly on the classification task, that is, to determine whether the client will subscribe to the term deposit.

Neural Network

Classification Report

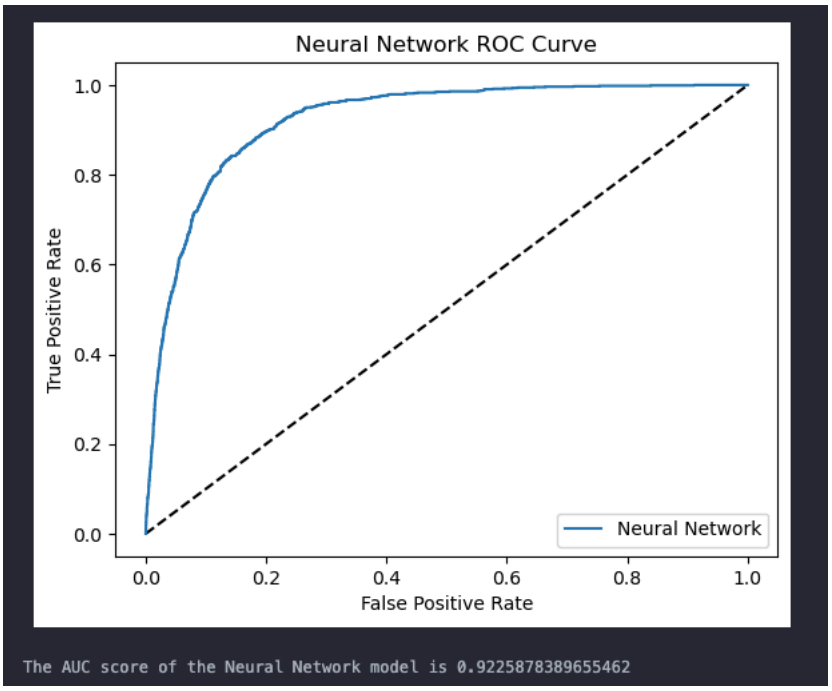
```
The accuracy of the Neural Network model is 0.9009178370009953
The confusion matrix of the Neural Network model is
[[7557  428]
 [ 468  590]]
The classification report of the Neural Network model is
              precision    recall  f1-score   support

           0       0.94      0.95      0.94      7985
           1       0.58      0.56      0.57      1058

    accuracy                           0.90      9043
   macro avg       0.76      0.75      0.76      9043
weighted avg       0.90      0.90      0.90      9043
```

The neural network model also achieved a very high accuracy value of 90% on the testing dataset, but due to the class imbalance in the testing dataset, this affects the results for class 1(“Yes”)’s

precision, recall, and F1-score. The result is perfect for the minority class when compared to the benchmarking values for an imbalanced dataset. (precision > 55, recall > 55 and f1-score > 50).

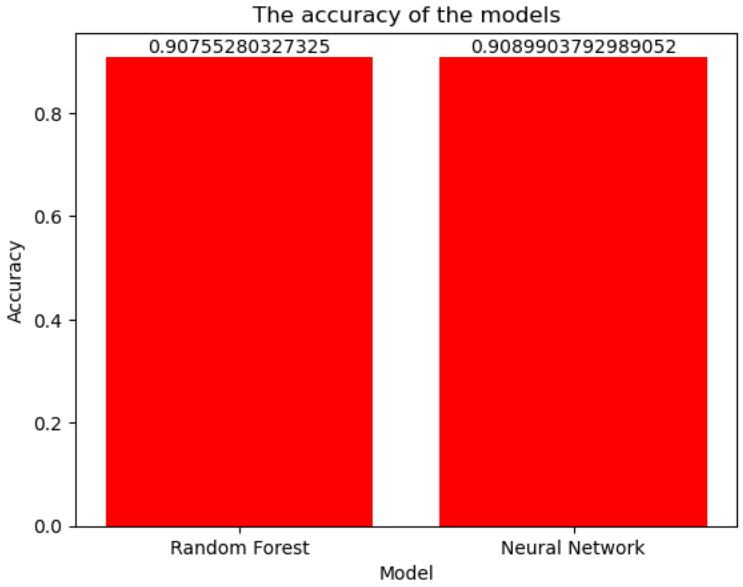
True Negative	7557
False Positive	428
False Negative	468
True Negative	590

ROC-AUC



The achieved AUC score of 92.2% indicates strong performance in the classification task on the highly imbalanced "Bank Marketing" dataset. This highlights the model's particular ability to correctly identify the difference between the two classes.

Experimental Results

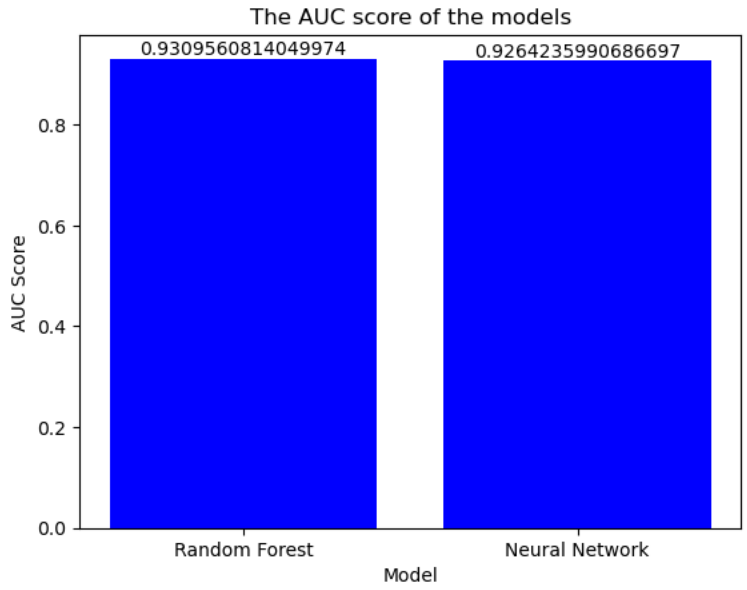


While both models have really good accuracy scores on the test set, the random forest model marginally outperformed the neural network model in the bank marketing classification task.

Metric	RF (class 1)	NN (class 1)	RF (class 0)	NN (class 1)
Precision	0.63	0.58	0.94	0.94
Recall	0.52	0.56	0.96	0.95
F1-score	0.57	0.57	0.95	0.94

Both models face challenges in classifying the minority class with the same f1-scores, The random forest model has a slightly high precision score, indicating it generates fewer false positive cases. A neural network identifies more true positive cases compared to random forest but at the cost of more false positives.

Both models perform extremely well on the majority class with similar precision, recall, and f1-score values above 0.94.



ROC-AUC scores demonstrate the model's ability to discriminate between the two classes over all cases. Both values are over 90%, indicating their strong performance in the bank marketing classification task.

In conclusion, both models perform well on the classification task, while the random forest model slightly outperforms the neural network in most metrics. It is evident that the random forest model performs better than the neural network, but the neural network remains a competitive alternative.

Limitations

Model Limitation

The random forest model is robust and accurate, but it can be prone to overfitting, especially when the number of trees and depth are not tuned carefully, which can lead to performance reduction on unseen data. Random forest is a complex combination of decision trees, and due to that reason, the interpretability is less.

The neural network is a great model for capturing non-linear patterns, but it suffers from high computational complexity, and its black-box nature and complexity limit the interpretability. In scenarios where the dataset is imbalanced, the results can look skewed. The hyperparameter tuning of a neural network can be challenging and time-consuming.

Data Limitation

The “Bank Marketing” dataset suffers from a huge class imbalance, with the majority class (nonsubscribers) and the minority class (subscribers). This can lead both the models to give skewed prediction results, this is imminent with lower recall for minority class, as observed in both models. Despite using dataset balancing techniques, the imbalance in the dataset still poses challenges.

Further Enhancements

Exploring more advanced feature engineering techniques such as domain-specific transformation to capture more hidden patterns in the data. Additionally improving the feature selection techniques can improve the model performance by reducing noise.

Adding more advanced ways to balance the dataset can lead to more generalized models.

Creating a more efficient data pipeline to effectively check the model’s performance on different feature sets and also to effectively tune model parameters that can lead to higher precision, recall, and f1-scores on minority classes.

References

GeeksforGeeks. (2022). *Compute Classification Report and Confusion Matrix in Python*. [online] Available at: https://www.geeksforgeeks.org/compute-classification-report-and-confusion-matrix-in-python/ .

Koehrsen, W. (2018). *Hyperparameter Tuning the Random Forest in Python*. [online] Medium. Available at: https://towardsdatascience.com/hyperparameter-tuning-the-random-forest-in-python-using-scikit-learn-28d2aa77dd74 .

www.youtube.com. (n.d.). *Krish Naik - YouTube*. [online] Available at: https://www.youtube.com/channel/UCNU_lfiiWBdtULKOw6X0Dig [Accessed 22 Dec. 2024].

Uci.edu. (2014). *UCI Machine Learning Repository*. [online] Available at: https://archive.ics.uci.edu/dataset/222/bank%2Bmarketing .

Python, R. (n.d.). *Visualizing Data in Python With Seaborn – Real Python*. [online] realpython.com. Available at: https://realpython.com/python-seaborn/ .

Chan, C. (2018). *What is a ROC Curve and How to Interpret It*. [online] Displayr. Available at: https://www.displayr.com/what-is-a-roc-curve-how-to-interpret-it/ .

Appendix

```
# import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# load the data
df = pd.read_csv('bank-full.csv', delimiter=';') # use the delimiter to separate the
columns as the data is separated by ';'
df.head()
```

```
# identify the number of rows and columns(features) in the dataset
print("The number of rows in the dataset is", df.shape[0])
print("The number of columns (features) in the dataset is", df.shape[1])
```

```
# identify the data types of the columns(features)
print("The data types of the columns (features) in the dataset are:")
print(df.dtypes)
```

```
# identify the number of missing values in the dataset
print("The number of missing values in the dataset is:")
print(df.isnull().sum())
```

```
# identify the number of duplicate rows in the dataset
print("The number of duplicate rows in the dataset is:", df.duplicated().sum())
# indentify the categorical features in the dataset
print("The categorical features in the dataset are (except the target column (y)):")
categorical_features = df.select_dtypes(include=['object']).columns
categorical_features = [feature for feature in categorical_features if feature !=
'y']
print(categorical_features)
```

```
# identify the unique values in the categorical features
print("The unique values in the categorical features are:")
for feature in categorical_features:
    print(feature)
    print(df[feature].unique())
    print(f"The number of unique values in the feature {feature} is
{len(df[feature].unique())}")
    print('-----------------------------------')
```

```
# identify the distribution of the categorical features
for feature in categorical_features:
    # order the categories based on their count
    order = df[feature].value_counts().index
    if feature == 'job':
        sns.countplot(y=feature, data=df, hue=feature, palette='viridis',
order=order)
        plt.title(feature)
        plt.show()
        # indentify the distribution of the categorical features with respect to the
target column
        sns.countplot(y=feature, data=df, hue='y', palette='viridis', order=order)
        plt.title(feature)
        plt.show()
        # check the distribution of the categorical features with respect to the
target column with value counts
        print(df.groupby([feature, 'y']).size().unstack().sort_values(by='yes',
ascending=False))
    else:
        sns.countplot(x=feature, data=df, hue=feature, palette='viridis',
order=order)
        plt.title(feature)
        plt.show()
        # check the distribution of the categorical features with respect to the
target column
        sns.countplot(x=feature, data=df, hue='y', palette='viridis', order=order)
        plt.title(feature)
        plt.show()
        # check the distribution of the categorical features with respect to the
target column with value counts order yes
        print(df.groupby([feature, 'y']).size().unstack().sort_values(by='yes',
ascending=False))
```

```
# identify the numerical features in the dataset
print("The numerical features in the dataset are:")
numerical_features = df.select_dtypes(include=['int64']).columns
```

```
print(numerical_features)
```

```
# identify the descriptive statistics of the numerical features (at once)
print("The descriptive statistics of the numerical features are:")
print(df[numerical_features].describe())
```

```
# indentify the distribution of the numerical features
for feature in numerical_features:
    sns.histplot(df[feature], kde=True, color='blue')
    plt.title(feature)
    plt.show()
    # display the discriptive statistics for each numerical feature
    print(df[feature].describe())
    # identify the distribution of the numerical features with respect to the target
column
    sns.boxplot(x='y', y=feature, data=df, hue='y', palette='viridis')
    plt.title(feature)
    plt.show()
    print('-----------------------------------')
```

```
# identify the distribution of the target column
print("The distribution of the target column (y) is:")
print(df['y'].value_counts())
sns.countplot(x='y', data=df, hue='y', palette='viridis')
plt.title('The distribution of the target column (y)')
plt.show()
```

```
# encode the target column
df['y'] = df['y'].apply(lambda x: 1 if x == 'yes' else 0)
```

```
# indentify the correlation between the numerical features and the target column (y)
before removing the outliers
correlation = df.corr()
sns.heatmap(correlation, annot=True, cmap='viridis')
plt.title('The correlation before removing the outliers')
plt.show()
```

```
# how many value are there in previous column that are latger than 0
print("The number of values in the 'previous' column that are larger than 0 is:",
df[df['previous'] > 0].shape[0])
```

```
# how many values are tere in duration column that are larger than 4000
print("The number of values in the 'duration' column that are larger than 2000 is:",
df[df['duration'] > 2000].shape[0])
```

```
# how many values are there in campaign column that are larger than 50
print("The number of values in the 'campaign' column that are larger than 50 is:",
df[df['campaign'] > 50].shape[0])
```

```
# take a copy of the dataset to handle the dataset
df2 = df.copy()
```

```
# handle the outliers in the numerical features using the winsorization method
from scipy.stats.mstats import winsorize
```

```
# apply the winsorization method to the numerical features with relevant limits
df2['campaign'] = winsorize(df2['campaign'], limits=[0, 0.005])
df2['balance'] = winsorize(df2['balance'], limits=[0, 0.01])
df2['duration'] = winsorize(df2['duration'], limits=[0, 0.005])
df2['pdays'] = winsorize(df2['pdays'], limits=[0, 0.01])
df2['previous'] = winsorize(df2['previous'], limits=[0, 0.01])
```

```
# indentify the distribution of the numerical features and the target column (y)
after removing the outliers
correlation = df2.corr()
sns.heatmap(correlation, annot=True, cmap='viridis')
plt.title('The correlation after removing the outliers')
plt.show()
```

```
# apply ordianl encoding to the categorical features (education) because it has an
ranking order
education_grading = {'unknown': 0, 'primary': 1, 'secondary': 2, 'tertiary': 3}
df2['education'] = df2['education'].map(education_grading)
```

```
# apply one-hot encoding to the categorical features because they don't have an
ranking order
df2 = pd.get_dummies(df2, columns=['job', 'marital', 'default', 'housing', 'loan',
'contact', 'month', 'poutcome'], drop_first=True)
```

```
# split the dataset into test and train datasets
from sklearn.model_selection import train_test_split

X = df2.drop('y', axis=1)
y = df2['y']
# stratify the target column to ensure that the distribution of the target column is
the same in both the train and test datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y,
random_state=42)
```

```
# scale the numerical features using the standard scaler
from sklearn.preprocessing import StandardScaler

scalerObj = StandardScaler()
X_train[numerical_features] = scalerObj.fit_transform(X_train[numerical_features])
X_test[numerical_features] = scalerObj.transform(X_test[numerical_features])
```

```
# check the shape of the train and test datasets
print("The shape of the train dataset is", X_train.shape)
print("The shape of the test dataset is", X_test.shape)
```

```
# GridSearchCV to find the best hyperparameters for the models
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Random Forest Classifier
from sklearn.ensemble import RandomForestClassifier

GridRF = RandomForestClassifier(random_state=42)
# define the hyperparameters to search
parameter_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'class_weight': ['balanced', 'balanced_subsample']
}
# apply the GridSearchCV to find the best hyperparameters
grid_search_rf = GridSearchCV(estimator=GridRF, param_grid=parameter_grid, cv=3,
n_jobs=-1, verbose=2)
grid_search_rf.fit(X_train, y_train)
```

```
print("The best hyperparameters for the Random Forest Classifier are:")
print(grid_search_rf.best_params_)
print("The best score for the Random Forest Classifier is:",
grid_search_rf.best_score_)
print("The best estimator for the Random Forest Classifier is:",
grid_search_rf.best_estimator_)
```

```
# build the Raaandom Forest model
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
# use the best hyperparameters to build the model
RandomFroestModel = RandomForestClassifier(random_state=42, n_estimators=300,
max_depth=30, class_weight='balanced_subsample', min_samples_split=5)
RandomFroestModel.fit(X_train, y_train)
# predict the target column
y_predRF = RandomFroestModel.predict(X_test)

print("The accuracy of the Random Forest model is", accuracy_score(y_test,
y_predRF))
print("The confusion matrix of the Random Forest model is")
print(confusion_matrix(y_test, y_predRF))
print("The classification report of the Random Forest model is")
print(classification_report(y_test, y_predRF))
```

```
# Evaluate the random forest model using ROC curve
from sklearn.metrics import roc_curve, roc_auc_score
# predict the probability of the target column
y_pred_probRF = RandomFroestModel.predict_proba(X_test)[:, 1]
fpr, tpr, thresholds = roc_curve(y_test, y_pred_probRF)

plt.plot([0, 1], [0, 1], 'k--')
plt.plot(fpr, tpr, label='Random Forest')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Random Forest ROC Curve')
plt.legend()
plt.show()

# calculate the AUC score
print("The AUC score of the Random Forest model is", roc_auc_score(y_test,
y_pred_probRF))
```

```
# balance the dataset using the SMOTE technique
from imblearn.over_sampling import SMOTE
```

```
smoteObj = SMOTE(random_state=42)
X_train_smote, y_train_smote = smoteObj.fit_resample(X_train, y_train)
```

```
# build the neural network model from tensorflow and keras
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.callbacks import EarlyStopping
# build the neural network model
nn_model = keras.Sequential([
    layers.Dense(100, activation='relu', input_shape=[X_train.shape[1]]),
    layers.Dense(50, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
# compile the model
nn_model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
# fit the model
early_stopping = EarlyStopping(monitor='val_loss', patience=5,
restore_best_weights=True)
nn_model.fit(X_train, y_train, epochs=50, batch_size=10, validation_split=0.2,
callbacks=[early_stopping])
# predict the target column
y_predNN = nn_model.predict(X_test)
y_predNN = [1 if x > 0.5 else 0 for x in y_predNN]

print("The accuracy of the Neural Network model is", accuracy_score(y_test,
y_predNN))
print("The confusion matrix of the Neural Network model is")
print(confusion_matrix(y_test, y_predNN))
print("The classification report of the Neural Network model is")
print(classification_report(y_test, y_predNN))
```

```
# evaluate the model using the ROC curve
y_pred_probNN = nn_model.predict(X_test)
fpr, tpr, thresholds = roc_curve(y_test, y_pred_probNN)

plt.plot([0, 1], [0, 1], 'k--')
plt.plot(fpr, tpr, label='Neural Network')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Neural Network ROC Curve')
plt.legend()
plt.show()
```

```
# calculate the AUC score
print("The AUC score of the Neural Network model is", roc_auc_score(y_test,
y_pred_probNN))
```

```
# plot the accuracy of the models
accuracy = [accuracy_score(y_test, y_predRF), accuracy_score(y_test, y_predNN)]
model = ['Random Forest', 'Neural Network']

plt.bar(model, accuracy, color='red')
# add the accuracy values on the bars
for i in range(len(accuracy)):
    plt.text(i, accuracy[i], accuracy[i], ha='center', va='bottom')
plt.xlabel('Model')
plt.ylabel('Accuracy')
plt.title('The accuracy of the models')
plt.show()
```

```
# plot the AUC score of the models
auc_score = [roc_auc_score(y_test, y_pred_probRF), roc_auc_score(y_test,
y_pred_probNN)]
model = ['Random Forest', 'Neural Network']

plt.bar(model, auc_score, color='blue')
# add the auc score values on the bars
for i in range(len(auc_score)):
    plt.text(i, auc_score[i], auc_score[i], ha='center', va='bottom')
plt.xlabel('Model')
plt.ylabel('AUC Score')
plt.title('The AUC score of the models')
plt.show()
```